

Tasklet Product Requirements Document

Version 0.1 | Author: Product | Status: Draft

Tasklet is a single-user task tracking app intended to demo the WorkspaceAI shell. It is intentionally minimal: a list of tasks with add, complete, and delete operations, plus filtering by completion state. The aim is to be small enough to read end to end in a single sitting, but realistic enough that a viewer can imagine extending it.

This document captures the goals, requirements, and milestones for the first cut. It is the source of truth for what version 0.1 must do. Future versions will live in their own PRDs.

1. Goals and Non-Goals

Goals

- Demonstrate end-to-end CRUD through a small typed API.
- Provide a recognizable UX (todo-app shape) so reviewers can evaluate code quality without learning a new product first.
- Keep the codebase under ~500 lines so it fits comfortably in a review session.

Non-goals

- Authentication, multi-user support, or sharing.
- Persistence beyond process lifetime. The store is in-memory.
- Mobile-specific layouts. Desktop browsers only.

2. Functional Requirements

F1. List tasks

The home screen displays all tasks in creation order. Each row shows the title, a completion checkbox, and a delete button. Completed tasks render with a strikethrough.

F2. Create task

A single-line input above the list accepts a task title. Empty titles are rejected. On submit, the task is appended and the input is cleared. The API returns the created task and the UI inserts it without a refetch.

F3. Toggle completion

Clicking the checkbox toggles the completed flag on the server and updates the row in place. Failure surfaces an inline error.

F4. Delete task

Clicking the delete button removes the task immediately. There is no undo in v0.1.

F5. Filter by status

Three tabs above the list All, Active, Done restrict which tasks are shown. The selection is local UI state; it does not persist across reloads.

3. Non-Functional Requirements

Performance

All operations complete in under 100 ms on localhost. The UI never blocks on a network call longer than 500 ms without a loading indicator.

Reliability

API errors are caught and shown inline. The frontend never leaves the user in an empty broken state at minimum, the last-known good list is rendered with an error banner.

Code quality

TypeScript strict mode is on for both projects. There are no any types. Components are < 100 lines each. The backend has no global mutable state outside the in-memory store.

4. Milestones

M1 API skeleton

Express app with health and tasks routes, in-memory store, and request logging. Manually verified with curl.

M2 UI skeleton

React app with the three components: NewTaskForm, FilterBar, TaskList. Wired to the backend via a small fetch wrapper.

M3 Polish

Error states, empty states, basic styling, README updates. At this point the demo is ready to show.

Out of scope for v0.1

Drag-to-reorder, due dates, tags, search, keyboard shortcuts, a real database, deployment.